Local AI Coding Agent in VS Code: Complete Setup Guide

Recommended stack: Cline (VS Code extension) + qwen/qwen3-30b-a3b-thinking-2507 (Q4_K_M)
Inference backend: LM Studio Bionic Local Model API at http://localhost:1235/v1 (llama.cpp, Vulkan)
Environment: Windows 11 (no WSL2) | AMD Radeon RX 9060 XT, 16 GB VRAM | Local git + LaTeX Workshop | November 4, 2026 | Prepared for Reik Schottstedt
Update of November 4, 2026: Step 3 now uses the OpenAI Compatible provider (validated on this machine); Step 7 adds the GitHub push workflow with web sign-in.

Overview of the Recommended Stack

Component	Choice	Rationale
VS Code extension	Cline	The only maintained open-source VS Code agent with a compact-prompt mode for local models, a visible Thinking section, and granular auto-approval up to full YOLO mode. Roo Code was archived in May 2026; Kilo Code works but adds multi-agent features a beginner does not need.
Model	qwen/qwen3-30b-a3b-thinking-2507	Always-thinking reasoning model explicitly flagged 'Trained for tool use' in the LM Studio catalog; 30.5B total parameters with only 3.3B active per token, so it runs acceptably even when part of the weights sit in system RAM.
Quantization	Q4_K_M GGUF (approx. 18.6 GB)	The practical 4-bit floor recommended for coding agents; Q3 quants measurably degrade tool-call reliability. On 16 GB VRAM a small share of the experts is held in system RAM; LM Studio lists 17 GB minimum system memory.
Endpoint	http://localhost:1235/v1	Your existing LM Studio Bionic Local Model API with just-in-time model loading; the Vulkan backend of llama.cpp serves the Radeon GPU.
Provider setting in Cline	OpenAI Compatible	Validated configuration on this machine (November 4, 2026). The dedicated 'LM Studio' provider fails against Bionic with an empty-model validation error (known issue cline#10016); see Step 3 and Step 8.
Permissions	Fully automatic (YOLO Mode)	Per your preference; safety net provided by Cline checkpoints plus the local git repository (see Sections 4 and 5). Git commits are automatic; pushes to GitHub run after a one-time web sign-in (Step 7).

Important: Why not gpt-oss-20b? Although it fits fully within 16 GB VRAM and is a strong native tool caller, it is unusable with Cline: Cline drives models with its own XML tool-call format rather than native function calling, and gpt-oss answers in its native harmony format. The Cline team confirmed gpt-oss models 'are terrible in Cline', and AMD's Cline + LM Studio model testing excluded gpt-oss-20b for broken tool behavior. Qwen3-30B-A3B-Thinking-2507 is the model that satisfies all three of your requirements (thinking, tool calling, local git commits) at this VRAM tier.

Prerequisites Checklist

- Windows 11; AMD Radeon RX 9060 XT (16 GB VRAM); at least 32 GB system RAM recommended (the model spills 2-3 GB into system RAM at Q4_K_M).

- LM Studio Bionic installed, Local Model API running at http://localhost:1235/v1, just-in-time model loading enabled (as per your setup).
- VS Code with LaTeX Workshop and a local git repository at D:\E\Wissensbasis\Projekte\Cheatsheet_collection.
- pdflatex available on the Windows PATH (MiKTeX or TeX Live); verify with pdflatex --version in PowerShell.
- Git for Windows with Git Credential Manager (bundled by default) for the GitHub push in Step 7.
- Approximately 19 GB free disk space for the model download.

Step 1: Configure LM Studio Bionic

1.1 Download the model

1. In Bionic, open Settings -> Local Models -> Explore (in classic LM Studio this tab is called Discover).
2. Search for 'qwen3-30b-a3b-thinking-2507' and select the entry qwen/qwen3-30b-a3b-thinking-2507 (GGUF, publisher lmstudio-community).
3. Choose the Q4_K_M quantization (approximately 18.6 GB) and confirm the download. This is the only model you need.
4. Ignore any device-fit warning that the file exceeds 16 GB VRAM: only 3.3B parameters are active per token, so a partial hold in system RAM costs little speed.

1.2 Set the model default load configuration (critical)

> *Known issue: just-in-time loading defaults to a 16,384-token context window, while Cline's system prompt including tool definitions already consumes roughly 25,000 tokens. With the default you will see 'context window exceeded' errors or a model that loses the task after two or three tool calls. Fix this once, permanently:*

1. Open My Models (sidebar) and find the card for qwen/qwen3-30b-a3b-thinking-2507.
2. Click the gear icon on the model card to open the default load parameters.
3. Set Context Length to 32768 and GPU offload to the maximum suggested value, then save. These parameters apply to every future JIT load.

Alternative via the lms command line (Bionic 1.1.8 restored lms compatibility):

```
lms unload qwen/qwen3-30b-a3b-thinking-2507
lms load qwen/qwen3-30b-a3b-thinking-2507 --context-length 32768
lms ps
```

Leave KV cache quantization disabled: it can persist context between tasks and produce unpredictable agent behavior; Cline's own LM Studio guide recommends keeping it off.

1.3 Verify the server settings

Setting (Server Settings / Developer)	Value	Reason
Server Port	1235	Your Bionic setup; Cline default is 1234 and must be corrected in Step 3.
Require Authentication	OFF	Keep OFF for now; the placeholder API key in Step 3 suffices. If you enable it later, enter the real Bionic API token in Cline's API Key field (the OpenAI Compatible provider has such a field).
Serve on Local Network	OFF	Nothing outside this machine should reach the endpoint.

Setting (Server Settings / Developer)	Value	Reason
Enable CORS	OFF	CORS is only required by browser-based clients (for example Open WebUI); the VS Code extension does not need it.
Just in Time Model Loading	ON	Keeps your existing setup; the load defaults from 1.2 apply.
Auto Unload Unused JIT Models	your choice	ON saves VRAM between sessions; OFF keeps the model resident and avoids reload times.

1.4 Enable reasoning separation

In the Developer settings, enable 'When applicable, separate reasoning_content and content in API responses'. This returns the model's chain of thought in the dedicated reasoning field of the OpenAI-compatible API, which Cline renders as the collapsible Thinking section instead of mixing reasoning into the visible answer.

1.5 Verify the endpoint

In PowerShell, run the following and confirm the model id appears:

```
curl http://localhost:1235/v1/models
```

Expected response (excerpt):

```
{ "data": [ { "id": "qwen/qwen3-30b-a3b-thinking-2507", "object": "model" } ] }
```

Note: Copy the exact 'id' string. If it deviates from the name above (for example it includes a quantization suffix), use the exact string as the Model ID in Cline in Step 3.

Step 2: Install the Cline Extension in VS Code

1. Open VS Code, then Extensions (Ctrl+Shift+X).
2. Search for 'Cline'; install the extension by publisher Cline Bot Inc. (5M+ installs, Apache 2.0). Reload VS Code when prompted.
3. Open your project: File -> Open Folder -> D:\E\Wissensbasis\Projekte\Cheatsheet_collection.
4. The Cline icon appears in the Activity Bar on the left; click it to open the Cline panel.

Step 3: Connect Cline to the Bionic Endpoint (Validated Configuration)

The following provider configuration was validated on your machine on November 4, 2026. It uses Cline's generic OpenAI-compatible provider, not the dedicated LM Studio provider; the reason is given below.

1. In the Cline panel, open Settings via the gear icon.
2. API Provider: select 'OpenAI Compatible'.
3. Base URL: enter http://localhost:1235/v1 (keep the /v1 suffix; it is required).
4. API Key: enter any non-empty placeholder, for example 'bionic'. Cline validates that the field is non-empty; Bionic does not check the value because the server runs without authentication.
5. Model ID: type or paste the exact id from Step 1.5, for example qwen/qwen3-30b-a3b-thinking-2507. Type it manually even if a dropdown appears; do not leave it empty. The first request triggers the JIT load, which takes 20-60 seconds.
6. Stay in Cline Settings, open the Features section, and enable 'Use Compact Prompt'.

Why the OpenAI Compatible provider and not 'LM Studio': Cline's dedicated LM Studio provider expects LM Studio's native REST route (/api/v0) for its model list. Bionic serves the OpenAI-compatible API at

/v1 on port 1235, so the provider finds no model there; the model field stays empty and Cline aborts every task at start with the validation error 'Failed to start task: [... 'path': ['model'], 'message': 'Too small: expected string to have >=1 characters'] ...' (known issue cline#10016; observed on your machine on November 4, 2026). The OpenAI Compatible provider talks to your /v1 endpoint directly and avoids this route entirely.

Why 'Use Compact Prompt' matters: it reduces Cline's system prompt to roughly one tenth of its full size, which is decisive for a local model with a 32k context. The trade-off disables MCP tools, Focus Chain and MTP features; none of these are needed for editing .tex files, compiling and committing.

> *Discouraged fallback: the provider 'LM Studio' with Base URL http://localhost:1235/v1 only becomes relevant if a future Cline release fixes the model resolution against Bionic. Until then it produces the 'Too small' error described above. If you ever enable 'Require Authentication' in Bionic, replace the placeholder API key with the actual Bionic API token; nothing else changes.*

Step 4: Permissions for Fully Automatic Operation

4.1 Granular option: Auto Approve

In the Cline settings panel, locate the Auto Approve section and enable the following toggles:

- Read project files: ON
- Edit project files: ON
- Execute safe commands: ON
- Execute all commands: ON

The distinction matters: git status, ls or cat count as safe commands, while pdflatex runs and git commit are classified as approval-required commands. 'Execute all commands' is therefore the toggle that makes compiling and committing fully automatic.

4.2 Single-switch option: YOLO Mode

Alternatively, in Cline Settings -> Features, check 'YOLO Mode'. This auto-approves every action: all file operations, all terminal commands including destructive ones, browser actions, MCP tools and mode transitions. This matches your stated preference for fully automatic operation; combine it with the safety measures below.

4.3 Safety measures for unattended operation

- The local git repository is the primary safety net: every committed state can be restored (git log, git checkout).
- Cline creates a checkpoint at every step; use the Compare and Restore buttons in the task header to roll back any change.
- Section 5 adds a project rule that commits locally after each task; pushes to GitHub are covered by Step 7 and require a one-time web sign-in.
- Watch the first few sessions completely before leaving tasks unattended.

Step 5: Project Rules for the Cheatsheet Repository

Create the file D:\E\Wissensbasis\Projekte\Cheatsheet_collection\.clinerules with the following content. Cline reads it automatically at the start of each task:

```
This repository contains LaTeX cheatsheets compiled with pdflatex.

- After each completed change to a .tex file, compile it with pdflatex
  and fix any LaTeX errors before finishing the task.
- After a successful task, stage the changed files with git add, create a
```

```
    local git commit with a concise, descriptive message, and push the
    commit to the origin remote on GitHub (after the one-time sign-in
    described in the setup guide, pushes are automatic).
  - Keep edits minimal; preserve the existing LaTeX preamble and
    formatting conventions.
  - Never publish secrets or personal data; the public cheatsheets
    repository must contain sanitized versions only.
```

Rules can be toggled per session in the rules popover below the Cline chat input.

Step 6: Verification of the Three Requirements

Test 1: Thinking (reasoning mode)

Prompt in the Cline chat: 'Think step by step: what is the sum of the integers from 1 to 100, and why?'

- Pass criterion: a collapsed 'Thinking' section appears above the answer; click it to inspect the chain of thought.
- If the reasoning text appears inside the answer instead, re-check Step 1.4 (reasoning separation).

Test 2: Terminal execution (compile a cheatsheet)

Prompt: 'Compile <your-sheet>.tex with pdflatex and report whether the PDF was produced and how many pages it has.' (Replace <your-sheet> with an actual file name in the repository.)

- Pass criterion: Cline runs pdflatex in the VS Code terminal without any approval dialog, reads the output, and reports the result; the corresponding .pdf file exists in the repository folder.
- The first run includes the JIT model load (20-60 seconds). Subsequent runs start immediately.

Test 3: Automatic git commit

Prompt: 'Stage the current changes and commit them with a descriptive message.'

- Pass criterion: no approval prompt appears; afterwards run in the VS Code terminal:
- git log --oneline -3 -> the new commit is at the top
- git status -> working tree clean (or only intended untracked files)
- The commit is local; pushing to GitHub is set up and verified in Step 7.

Context check

While Cline works, watch the token bar at the top of the task view: total usage should stay clearly below 32,768 tokens. If it approaches the limit, start a new task (plus button in the task header) rather than letting the context overflow.

Step 7: Push to GitHub via Web Sign-In

7.1 One-time preparation: create the remote and connect it

1. In your browser, sign in at github.com (account ReikS) and create the target repository, for example ReikS/cheatsheets for the sanitized public cheatsheets; keep sensitive personal material in your private repository, as before.
2. In the VS Code terminal, inside D:\E\Wissensbasis\Projekte\Cheatsheet_collection, connect the local repository and perform the first push:

```
git remote add origin https://github.com/ReikS/cheatsheets.git
git push -u origin main
```

3. This first push triggers the sign-in described in 7.2 or 7.3. After it succeeds, the local branch main tracks origin/main.

7.2 What happens on the first push: browser sign-in via Git Credential Manager

Git for Windows ships with Git Credential Manager (GCM); its credential helper is configured as 'manager' by default, and its default credential store on Windows is Windows Credential Manager.

1. On the first HTTPS push, GCM opens your default browser with the GitHub sign-in page. GCM may first show a chooser: 1. Web browser (default), 2. Device code, 3. Personal access token. Choose Web browser.
2. If you are already signed in to GitHub in that browser, authorization completes with one confirmation; otherwise enter your GitHub credentials and complete two-factor authentication.
3. After the browser confirms, GCM stores the OAuth token in Windows Credential Manager. Every later push, including pushes triggered by Cline in the terminal, is then automatic and needs no interaction.

Optional settings to always prefer the browser flow and suppress the GUI chooser:

```
git config --global credential.gitHubAuthModes browser
git config --global credential.guiPrompt false
```

Verification: the repository page on github.com shows the branch main and the pushed commit.

7.3 Alternative without a browser redirect: device code flow

If the browser cannot open on your machine, choose 'Device code' in the GCM chooser (or run a push in an environment where GCM falls back to device mode). GCM prints a URL and a one-time code; open the URL on any device, sign in to GitHub, and enter the code. The token is then stored exactly as in 7.2. VS Code's own GitHub sign-in offers the same device flow: it shows a code and the page github.com/login/device where you enter it.

7.4 Alternative: VS Code's built-in GitHub authentication

VS Code has GitHub authentication built in; no extension is required. When a push needs credentials and VS Code handles the prompt, it shows 'Sign in to GitHub' and opens the browser authorization page ('Authorize Visual Studio Code'); after authorizing you return to VS Code. Git commands run in the integrated terminal, including those executed by Cline, are automatically authenticated with this account (VS Code setting git.terminalAuthentication, enabled by default). You can also sign in proactively: Command Palette (Ctrl+Shift+P) -> 'GitHub: Sign In'. The token is stored in Windows Credential Manager.

> *Both routes (GCM and VS Code) end with a credential in Windows Credential Manager. If pushes prompt repeatedly, remove the stale entry: Control Panel -> User Accounts -> Credential Manager -> Windows Credentials -> remove the entries beginning with 'git:' and push again.*

7.5 Letting Cline push

1. The .clinerules from Step 5 instruct Cline to push after each commit. Run the first push supervised: prompt Cline with 'Commit the current changes and push to origin.'
2. The first push opens the browser sign-in once (7.2 or 7.4). From then on, pushes are fully automatic.
3. Verification: git log origin/main lists the remote branch, and the GitHub repository page shows the commits.

Step 8: Troubleshooting Known Issues

Symptom	Cause	Fix

Symptom	Cause	Fix
Failed to start task with [... 'path': ['model'], 'message': 'Too small: expected string to have >=1 characters']; chat stuck at 'Thinking...'.	Dedicated LM Studio provider selected in Cline. It queries the native /api/v0 route, which Bionic does not serve, so the model field stays empty and Cline's validation rejects it (known issue cline#10016).	Switch API Provider to 'OpenAI Compatible', Base URL http://localhost:1235/v1, any non-empty API key, and type the exact Model ID from /v1/models (Step 3). Validated on this machine.
'Context window exceeded' or 'Cannot truncate prompt' errors; model forgets the task after 2-3 tool calls.	JIT default context of 16,384 tokens; Cline needs about 25k for its system prompt.	Set Context Length 32768 in the model default load parameters (Step 1.2); confirm 'Use Compact Prompt' is on.
HTTP 500 error on every Cline request.	Model ID mismatch between Cline and the endpoint.	Copy the exact id from http://localhost:1235/v1/models into Cline.
Cline stuck at 'API Request...'.	Server not running or model not loaded.	Check the server status in the Bionic Developer panel; re-run curl http://localhost:1235/v1/models.
Thinking text mixed into the normal answer.	Reasoning separation disabled.	Enable the separate reasoning_content setting (Step 1.4).
Generation is very slow.	Expert weights partially held in system RAM; low free system RAM.	Close memory-intensive applications; reduce Context Length to 24,576; as a last resort use a Q4_K_S or Q3_K_L quant (fully GPU-resident, but Q3 measurably degrades tool-call reliability).
Extension cannot connect although curl works.	Wrong Base URL in Cline.	Base URL must be http://localhost:1235/v1 including the /v1 suffix (Step 3).
Push fails with an authentication error or prompts every time.	Stale or missing credential in Windows Credential Manager.	Remove entries beginning with 'git:' in Credential Manager, then push again to trigger a fresh browser sign-in (Step 7.2 and 7.4).
Model produces broken or missing tool calls.	Substituted model not suited to Cline (for example gpt-oss-20b, DeepSeek-R1 distills, 7B-class models).	Use qwen/qwen3-30b-a3b-thinking-2507 as recommended; Cline demands reliable structured tool calls.

Key Sources

- Cline documentation, Run Cline with local models (Use Compact Prompt): docs.cline.bot/running-models-locally/overview
- Cline documentation, Auto Approve and YOLO Mode: docs.cline.bot/features/auto-approve
- Cline blog, 'Cline + LM Studio: the local coding stack' (context window, KV cache quantization off, 4-bit guidance): cline.bot/blog/local-models
- LM Studio documentation, Server Settings (Server Port, Require Authentication, Enable CORS, JIT loading): lmstudio.ai/docs/developer/core/server/settings
- LM Studio model page qwen3-30b-a3b-thinking-2507 (Trained for tool use; Reasoning; 17 GB minimum system memory): lmstudio.ai/models/qwen/qwen3-30b-a3b-thinking-2507
- LM Studio API changelog (separate reasoning_content setting): lmstudio.ai/docs/developer/api-changelog

- Bionic changelog 1.1.8 (lms compatibility with Bionic): lmstudio.ai/changelog/bionic-v1.1.8
- Cline issue tracker, issue #10016 (empty model id / 'Too small' validation failure with the LM Studio provider)
- aicoderscope, 'LM Studio Bionic as a Coding Agent in 2026' (Cline provider base URL, >25k context for agentic tools, missing API-key field cline#9668): aicoderscope.com/blog/lm-studio-bionic-coding-agent-privacy-guide-2026
- dev.to (Jovan Chan), 'Cline + LM Studio 2026: complete setup guide, the 32k context trap'
- VDF AI, 'Best Local LLM for Agents and Tool Calling' (October 2026 shortlist by VRAM tier): vdf.ai/blog/best-local-llm-for-agents-tool-calling
- modelfit, 'Best Local LLMs for Cline (2026 Tier List)' (tool-call reliability, Q4_K_M floor, Q3 degradation): modelfit.io/tools/cline
- Community reports on gpt-oss in Cline (r/CLine; AMD model-testing thread; Cline team statement on gpt-oss tool-call incompatibility)
- VS Code documentation, 'Work with GitHub in VS Code' (built-in GitHub authentication, browser sign-in on push): code.visualstudio.com/docs/sourcecontrol/github
- Git documentation, 'Git credential helpers' (Git Credential Manager included with Git for Windows; OAuth opens a browser; Windows Credential Manager storage): git-scm.com/doc/credential-helpers
- WindowsForum tutorial, 'Git for Windows: Reset Stale HTTPS Credentials' (helper name 'manager', default store wincredman, removing stale git: entries)
- Stack Overflow, Git Credential Manager auth-mode chooser (Web browser / Device code / PAT; credential.gitHubAuthModes browser; credential.guiPrompt false)

Guide compiled on October 29, 2026; updated November 4, 2026 (OpenAI Compatible provider validated; GitHub push workflow added). Tool versions move quickly; if a menu label deviates, the setting names above follow the official documentation as of this date.